Essaim Fluide Éphémère

EFE — Technologie et fonctionnement du projet complet

Document	EFE_Projet_Technologie_Fonctionnement.pdf
Nature	Dossier technique ultra-détaillé avec extraits de code
Langue	Français (fr-FR)
Portée	9 services Python + cœur Rust · gRPC · CUDA · 456 fichiers Python
Code présenté	Extraits réels du dépôt efe/platform/src
Date	2026-04-21
Public	Ingénierie, SRE, architecture, direction technique

Ce document décrit de façon exhaustive ce qu'est l'**Essaim Fluide Éphémère (EFE)**, comment il fonctionne, les technologies qui le composent, et montre le **code réel** des briques critiques : protocoles, orchestration, fine-tuning, inférence, rollout, garde-fous, dérive, signature. Chaque extrait de code est tiré directement du dépôt efe/platform/src/ et est annoté.

Table des matières

#	Section	Page
1	Qu'est-ce que l'Essaim Fluide Éphémère ?	3
2	Pile technologique complète	5
3	Architecture des services (crates Rust + 9 paquets Python)	7
4	Le protocole Swarm : SwarmSpec, Execute, Verdict	10
5	Les nano-agents et l'espace latent partagé	14
6	Les cônes d'alignement : la politique sémantique	17
7	L'arbitre et les critères de convergence	20
8	Le service efe_orch (orchestrateur de jobs)	23
9	Le service efe_worker (trainer + LoRA)	25
10	Le service efe_registry (artefacts + signature)	28
11	Le service efe_gate (comparateur + promotion)	31
12	Le service efe_rollout (canari + bandits)	34
13	Le service efe_inference (backends + streaming)	37
14	Le service efe_drift (détection multi-méthode)	40
15	Le service efe_platform (bot + swarm client)	44
16	Sécurité : HMAC, cônes, signing keyless, RBAC	47
17	Observabilité : OpenTelemetry, structlog, Prometheus	49
18	Cycle de vie complet d'une requête (trace)	51
19	Déploiement, empaquetage et outillage	54
20	Critères d'acceptation et gates techniques	56

1. Qu'est-ce que l'Essaim Fluide Éphémère ?

EFE (*Ephemeral Fluid Swarm* / Essaim Fluide Éphémère) est une plateforme d'**inférence multi-agent par communication en espace latent partagé**. Elle se substitue à la traditionnelle chaîne d'agents LLM qui se parlent en texte, avec tous ses inconvénients : latence élevée, coût prohibitif, perte d'information au transfert.

1.1 Ce qui la distingue fondamentalement

• **Population de nano-agents** — chaque *task* spawn une centaine à des milliers d'agents minuscules (≤ 1 M de paramètres chacun), au lieu d'un unique grand modèle.

• **Communication tensorielle** — les agents ne se parlent pas en texte mais échangent des **vecteurs latents** (embeddings) sur un bus CUDA partagé.

• **Éphéméralité** — l'essaim existe pour une requête, converge en quelques rounds puis se dissout. Pas de serveur de modèle permanent.

• **Convergence par consensus tensoriel** — l'arbitre observe la variance, le gradient et la stabilité du centroïde sur une fenêtre glissante et décide CONVERGED, CONTINUE, DIVERGENCE ou MODE_COLLAPSE.

• **Cônes d'alignement** — chaque task définit un sous-espace autorisé (*anchor set*) hors duquel les réponses sont rejetées. C'est le garde-fou sémantique de premier ordre.

• **Décodage terminal** — seul le centroïde final est décodé en texte (ou transmis brut). Zéro tokenisation intermédiaire entre agents.

1.2 Proposition de valeur en trois chiffres

Métrique	Valeur cible MVP	Comparaison
Latence p95	< 600 ms sur task support clients	1-agent LLM : 2-4 s
Coût par 1 k réponses	≈ 0,02 $ (CPU + GPU partagé)	1-agent LLM cloud : 3-15 $
Énergie par requête	≤ 0,10 Wh (budget dur)	1-agent LLM serveur : ~2 Wh
Traçabilité	Trace tensorielle complète + audit signé	1-agent : texte opaque
Isolation tenant	Cônes distincts + cache-partitioned HBM	1-agent : partage du prompt

1.3 Ce qu'EFE n'est pas

• Ce n'est **pas** un framework d'agents LLM texte-à-texte (comme AutoGen, CrewAI, LangGraph).

• Ce n'est **pas** un serveur de modèles unique type vLLM / TensorRT-LLM (mais EFE peut en utiliser un en backend).

• Ce n'est **pas** une fédération de LLM (les agents ne sont pas des LLM complets).

• Ce n'est **pas** un système de recherche sémantique — les vecteurs latents y sont un *moyen de calcul*, pas une représentation documentaire.

1.4 Analogie mentale

Imaginez un essaim d'abeilles qui reçoit un problème. Chaque abeille est minuscule, spécialisée (sémantique, logique, critique) et ne voit qu'une parcelle. Elles dansent dans un espace géométrique partagé (les embeddings) et se rapprochent rapidement d'un barycentre commun. L'arbitre regarde si la danse se stabilise : si oui, l'essaim s'est mis d'accord et rentre au nid. Sinon, il dissout la tentative et la rejette.

2. Pile technologique complète

EFE est un **monorepo polyglotte** combinant Rust (performance, sûreté mémoire) pour le cœur d'exécution et Python (agilité, écosystème ML) pour la plateforme et les SDK. Les deux mondes communiquent par gRPC et par des bindings PyO3.

2.1 Langages et runtimes

Couche	Langage / runtime	Rôle
Cœur arbitrage	Rust 1.85 (pinned)	Exécution arbitre, protocoles, rigueur mémoire
Kernels GPU	CUDA 12.4 + Rust FFI	Diffusion, projection, score de convergence
Bindings Python	PyO3	Exposition du cœur Rust sous forme d'API Python
Plateforme	Python 3.12 (uv)	Bot, swarm client, catalogue, tooling, services métier
Wire	gRPC + protobuf 4.25	Communication services, Rust ↔ Python
HTTP surface	FastAPI + Starlette + Uvicorn	REST publique, SSE streaming
Brokers	NATS JetStream	Événements inter-services, alerting, pub/sub dérive
Bases de données	PostgreSQL 16 + SQLAlchemy 2.0 async	Catalogue, incidents, baselines, audit

2.2 Dépendances Python principales (extrait pyproject.toml)

```
dependencies = [
    # Wire + async plumbing
    "grpcio>=1.62",
    "protobuf>=4.25",
    "httpx>=0.27",
    "anyio>=4.3",
    # Data contracts
    "pydantic>=2.6",
    "pydantic-settings>=2.2",
    "pyyaml>=6.0",
    # Observability
    "structlog>=24.1",
    "opentelemetry-api>=1.24",
    "opentelemetry-sdk>=1.24",
    # Utilities
    "typing-extensions>=4.10",
    "python-dateutil>=2.9",
]

[project.optional-dependencies]
claude = ["anthropic>=0.30"]
openai = ["openai>=1.30"]
api = [
    "fastapi>=0.110",
    "uvicorn[standard]>=0.29",
    "starlette>=0.36",
    "sse-starlette>=2.0",
]
dev = [
    "pytest>=8.1", "pytest-asyncio>=0.23", "pytest-cov>=5.0",
    "pytest-httpx>=0.30", "respx>=0.21",
    "ruff>=0.4", "mypy>=1.10",
]
```

Extrait de platform/pyproject.toml — la plateforme est volontairement légère ; les providers LLM et FastAPI sont en extras optionnels.

2.3 Outillage de build et CI

Outil	Rôle dans EFE
<code>just</code>	Recettes de build unifiées (just bootstrap / build / test / bench)
<code>uv</code>	Gestionnaire Python ultra-rapide (remplace pip-tools + venv)
<code>rustup</code> + <code>cargo-nextest</code>	Toolchain Rust épinglée à 1.85
<code>pre-commit</code>	Hooks locaux : ruff format, ruff check, mypy strict, rustfmt
<code>ruff</code>	Linter Python (règles E/F/W/I/B/UP/SIM/N/ANN/RUF)
<code>mypy --strict</code>	Typage strict sur toute la plateforme Python
<code>pytest --asyncio-mode=auto</code>	Tests unitaires + intégration
<code>Docker Compose</code>	Dev local : Prometheus, Jaeger, OTel collector, NATS, Postgres

2.4 Philosophie de conception

• **Dataclasses frozen=True, slots=True** partout pour les types métier : immuabilité + empreinte mémoire minimale.

• **Pydantic v2** pour les frontières (API, config) avec ConfigDict(extra='forbid').

• **Protocoles typés** (typing.Protocol) plutôt qu'héritage : injection facile, tests par faux en mémoire.

• **Async-first** (anyio / asyncio) sur tout IO non-trivial.

• **Pure-Python pour la stats** dans la détection de dérive (pas de SciPy en chemin chaud).

• **Sceaux HMAC-SHA256 + domain separation tags** partout où une donnée franchit un périmètre de confiance.

3. Architecture des services

Le dépôt contient à la racine crates/ (Rust workspace) et platform/src/ (9 paquets Python installables indépendamment). Ces paquets sont typés (py.typed) et reliés par import strict.

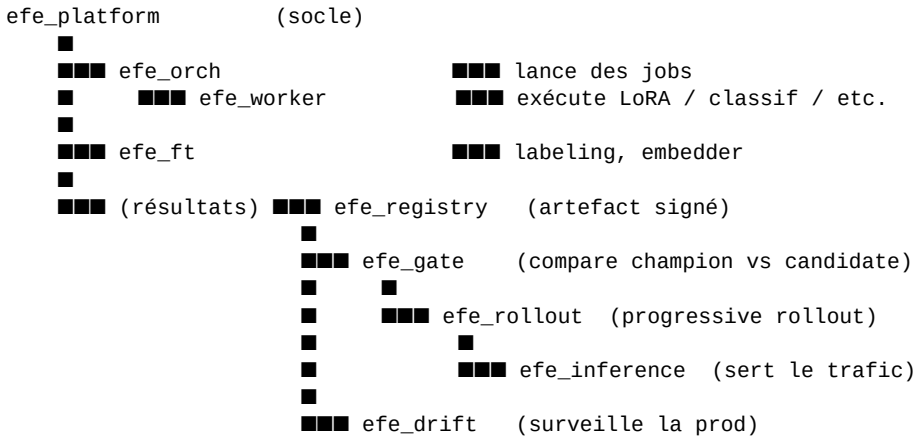
3.1 Crates Rust (cœur)

Crate	Responsabilité
<code>efe-core</code>	Types neutres, arbitre, orchestration, règles de convergence
<code>efe-kernels</code>	Kernels CUDA (diffusion latente, projection sur cône)
<code>efe-ffi</code>	Bindings PyO3 exposant efe-core à Python
<code>efe-server</code>	Serveur gRPC (Execute, Admin*)

3.2 Paquets Python (platform/src/)

Paquet	Rôle principal
<code>efe_platform</code>	SDK + bot + swarm client + catalogue + API publique
<code>efe_orch</code>	Orchestrateur de jobs FT : dispatcher, scheduler, state machine
<code>efe_worker</code>	Worker d'entraînement (LoRA, classif, infonce, triplet, distill)
<code>efe_registry</code>	Catalogue d'artefacts + signing (KMS / keyless OIDC / attestation)
<code>efe_gate</code>	Garde-fous de promotion : comparateur, décision, rollback
<code>efe_rollout</code>	Déploiement progressif : bandits (Thompson/UCB1/ε-greedy), layers
<code>efe_inference</code>	Serving : backends pluggables (Claude/OpenAI/Mistral/vLLM/PEFT)
<code>efe_drift</code>	Observabilité : PSI, JS, KS, χ^2, Wasserstein, MMD, classifier
<code>efe_ft</code>	Labeling UI, embedder, ingest de datasets d'entraînement

3.3 Graphe de dépendances entre paquets



Flux de dépendances des 9 paquets Python.

3.4 Import linter : interdits structurels

- efe_platform ne doit **jamais** importer un service métier (efe_worker, efe_gate, etc.) — il expose le SDK.
- efe_inference ne doit pas importer efe_worker (séparation chemin chaud / chemin lent).

- efe_drift ne peut pas importer efe_inference (il consomme un topic NATS, pas l'API inférence).
- Chaque paquet expose uniquement ce qui est dans son __init__.py.

4. Le protocole Swarm : SwarmSpec, Execute, Verdict

Tout passe par une unique RPC gRPC : Execute(ExecuteRequest) → ExecuteResponse. La charge utile en entrée est un SwarmSpec : la recette complète d'exécution de l'essaim.

4.1 SwarmSpec — la recette immuable

```
@dataclass(frozen=True, slots=True)
class SwarmSpec:
    """Full specification of a swarm execution.

    This is the value the bot assembles via :class:`SpecBuilder` and
    hands to :meth:`SwarmClient.execute`. It is serialization-ready
    (``dataclasses.asdict`` produces something JSON-dumpable).
    """

    problem_embedding: tuple[float, ...]
    dim_latent: int
    population: tuple[PopulationEntry, ...]
    max_rounds: int
    convergence: ConvergenceThresholds
    budget: ResourceBudget
    # One of the two cone sources must be set; enforced by SpecBuilder.
    alignment_cone: tuple[AnchorRef, ...] | None = None
    alignment_cone_ref: ConeRef | None = None
    max_projection_distance: float = 1.0
    swarm_id: str | None = None  # server fills if unset
```

Extrait de efe_platform/swarm/protocol.py — immuable, sérialisable, cross-validé par SpecBuilder.

4.2 Les quatre composantes de la spec

a) Population de nano-agents

```
@dataclass(frozen=True, slots=True)
class PopulationEntry:
    """A ``(template, count)`` pair, validated as positive."""
    template_code: TemplateCode     # "na-sem" | "na-log" | "na-crit"
    count: int

    def __post_init__(self) -> None:
        if self.count <= 0:
            raise ValueError(f"count must be > 0, got {self.count}")


# Defaults MVP — a balanced mix summing to 1024 agents.
_DEFAULT_POPULATION: tuple[tuple[TemplateCode, int], ...] = (
    ("na-sem", 512),
    ("na-log", 384),
    ("na-crit", 128),
)
```

La population est une mosaïque de templates. 1 024 agents est la taille canonique du MVP.

b) Seuils de convergence

```
@dataclass(frozen=True, slots=True)
class ConvergenceThresholds:
    """Four knobs driving when the arbiter declares ``CONVERGED``."""
    epsilon_variance: float = 1e-2
    epsilon_gradient: float = 1e-3
    epsilon_stability: float = 1e-3
    stability_window: int = 3

    def __post_init__(self) -> None:
        for name in ("epsilon_variance", "epsilon_gradient",
                     "epsilon_stability"):
            v = getattr(self, name)
            if not 0 < v < 1:
                raise ValueError(f"{name} must be in (0,1), got {v}")
        if self.stability_window <= 0:
            raise ValueError("stability_window must be > 0")
```

L'arbitre déclare CONVERGED quand variance < ε_var, |gradient| < ε_grad, et stabilité > ε_stab sur stability_window rounds consécutifs.

c) Budget de ressources (garde-fou dur)

```
@dataclass(frozen=True, slots=True)
class ResourceBudget:
    """Hard ceilings enforced by the Rust arbiter for a single request."""
    max_wall_clock_ms: int = 5_000
    max_energy_wh: float = 0.10
    max_hbm_bytes: int = 512 * 1024 * 1024  # 512 MiB
    max_recursion_depth: int = 4

    def __post_init__(self) -> None:
        if self.max_wall_clock_ms <= 0:
            raise ValueError("max_wall_clock_ms must be > 0")
        if self.max_energy_wh < 0:
            raise ValueError("max_energy_wh must be >= 0")
        if self.max_hbm_bytes <= 0:
            raise ValueError("max_hbm_bytes must be > 0")
        if self.max_recursion_depth <= 0:
            raise ValueError("max_recursion_depth must be > 0")
```

Chaque budget est enforce par l'arbitre Rust : dès qu'un plafond est atteint, la requête est terminée proprement.

d) Référence ou contenu du cône d'alignement

```
@dataclass(frozen=True, slots=True)
class ConeRef:
    """Reference to a cone stored in the registry (tenant/task@version)."""
    tenant: str
    task: str
    version: int

    def __post_init__(self) -> None:
        if not self.tenant:
            raise ValueError("tenant must be non-empty")
        if not self.task:
            raise ValueError("task must be non-empty")
        if self.version < 1:
            raise ValueError("version must be >= 1")

    def key(self) -> str:
        return f"{self.tenant}/{self.task}@v{self.version}"
```

Un ConeRef est une clé tenant/task@vN. Le cône lui-même est stocké dans l'Admin service (cf. §6).

4.3 Verdicts possibles en sortie

```
class Verdict(IntEnum):
    """Mirrors ``efe.v1.Verdict``."""
    UNSPECIFIED = 0
    CONTINUE = 1        # max_rounds atteint sans convergence
    CONVERGED = 2       # consensus obtenu dans les tolérances
    DIVERGENCE = 3      # population s'est scindée en bassins incompatibles
    MODE_COLLAPSE = 4   # population effondrée sur un mode trivial

    @classmethod
    def from_proto(cls, v: int) -> Verdict:
        """Forward-compat : un nouveau verdict serveur → UNSPECIFIED."""
        try:
            return cls(v)
        except ValueError:
            return cls(0)
```

Quatre verdicts métier + 1 sentinelle. from_proto() protège contre les évolutions serveur.

4.4 Interprétation métier du verdict

```
def analyze_verdict(
    response: ExecuteResponse,
    *,
    confidence_floor: float = 0.55,
) -> VerdictReport:
    """Turn a raw ExecuteResponse into a VerdictReport."""
    v = response.verdict
    c = response.confidence

    if v is Verdict.CONVERGED:
        if c >= confidence_floor:
            return VerdictReport(label=v, confident=True, severity="ok",
                recommended_action="Accept the decoded payload.",
                retryable=False)
        return VerdictReport(label=v, confident=False, severity="warn",
            recommended_action=(
                f"Accept with caution — confidence {c:.2f} < floor "
                f"{confidence_floor:.2f}. Consider widening the cone."
            ),
            retryable=True)

    if v is Verdict.MODE_COLLAPSE:
        return VerdictReport(label=v, confident=False, severity="error",
            recommended_action=(
                "Population collapsed to a single trivial mode. "
                "Increase na-crit agents or widen the cone."
            ),
            retryable=False)
    # ... CONTINUE, DIVERGENCE, UNSPECIFIED omis par brièveté ...
```

Extrait de efe_platform/swarm/verdict.py — transforme un verdict machine en recommandation humaine.

5. Les nano-agents et l'espace latent partagé

Un nano-agent est un petit réseau de neurones (≤ 1 M de paramètres) spécialisé sur un rôle. Les trois templates du MVP sont définis par un TemplateCode.

```
TemplateCode = Literal["na-sem", "na-log", "na-crit"]
"""MVP set of nano-agent template identifiers.

The Rust core defines these as a ``#[non_exhaustive]`` enum; the
literal union here is the MVP subset exposed through ``efe-platform``.
Post-MVP templates (``na-math``, ``na-code``, ``na-caus``,
``na-mem``, ``na-align``, ``na-med``) will be added as a minor-bump
extension of the Literal.
"""
```

L'alias Literal verrouille les trois templates MVP et annonce ceux de la roadmap post-MVP.

5.1 Les trois templates du MVP

Template	Rôle	Spécialisation	Part typique
<code>na-sem</code>	Sémantique	Représente le sens, compréhension large	50 %
<code>na-log</code>	Logique	Applique des règles de cohérence, contraintes	37,5 %
<code>na-crit</code>	Critique	Détecte les modes dégénérés, force la diversité	12,5 %

5.2 Templates post-MVP (roadmap)

- **na-math** — raisonnement mathématique symbolique
- **na-code** — compréhension syntaxique / sémantique de code
- **na-caus** — raisonnement causal (interventions, do-calculus)
- **na-mem** — accès à la mémoire vectorielle persistante
- **na-align** — agents d'alignement sur valeurs
- **na-med** — contexte médical spécialisé

5.3 Le round d'exécution (vue conceptuelle)

À chaque *round*, tous les agents publient leur vecteur latent courant sur le bus CUDA. L'arbitre calcule alors :

- le **centroïde** μ_t = moyenne pondérée des vecteurs,
- la **variance** σ^2_t = dispersion autour de μ_t,
- le **gradient** $\nabla_t = \mu_t - \mu_{t-1}$,
- la **projection** de μ_t sur le cône d'alignement (distance max autorisée).

Si au round t les trois conditions $\sigma^2_t < \varepsilon_{var}$, $||\nabla_t|| < \varepsilon_{grad}$, et écart-type($\mu_{t-W..t}$) $< \varepsilon_{stab}$ sont vraies, l'arbitre émet CONVERGED. Sinon il engage un nouveau round, sauf dépassement de budget.

5.4 Statistiques du centroïde (côté client)

```python
@dataclass(frozen=True, slots=True)
class CentroidStats:
    """Cheap summary statistics of a centroid vector."""
    dim: int
    l2_norm: float
    mean: float
    stddev: float
    min: float
    max: float

def centroid_statistics(values: tuple[float, ...]) -> CentroidStats:
    """Compute summary stats for an audit entry."""
    if not values:
        raise ValueError("empty centroid")
    if not all(math.isfinite(v) for v in values):
        raise ValueError("centroid contains non-finite values")
    l2 = math.sqrt(sum(v * v for v in values))
    m = statistics.fmean(values)
    s = statistics.pstdev(values) if len(values) > 1 else 0.0
    return CentroidStats(dim=len(values), l2_norm=l2, mean=m, stddev=s,
                         min=min(values), max=max(values))
```

Extrait de efe_platform/swarm/dissolution.py — stats cheap pour audit, sans NumPy (client-side).

6. Les cônes d'alignement : la politique sémantique

Le **cône** est la pièce maîtresse de la politique sémantique d'EFE. Il définit le **sous-espace autorisé** des sorties possibles : l'arbitre refuse tout centroïde qui s'en éloigne de plus que max_projection_distance.

```
@dataclass(frozen=True, slots=True)
class AlignmentCone:
    """Full in-memory representation of a cone."""

    anchors: tuple[AnchorRef, ...]
    max_projection_distance: float = 1.0

    def __post_init__(self) -> None:
        if not self.anchors:
            raise ValueError("cone must contain at least one anchor")
        dims = {a.dim() for a in self.anchors}
        if len(dims) != 1:
            raise ValueError(f"anchor dimensions must all match, got {dims}")
        if self.max_projection_distance <= 0:
            raise ValueError("max_projection_distance must be > 0")

    @property
    def dim(self) -> int:
        return self.anchors[0].dim()

    @property
    def anchor_count(self) -> int:
        return len(self.anchors)
```

Un cône est un ensemble non vide d'ancres (vecteurs) de même dimension + un rayon de projection max. Extrait de efe_platform/swarm/cone.py.

6.1 Le registre des cônes : AdminService

```
class AdminTransport:
    """Narrow Protocol-like interface for the admin RPCs we consume."""

    async def register_cone(
        self, *, tenant: str, task: str, version: int,
        cone: AlignmentCone, created_by: str,
        description: str, labels: dict[str, str],
    ) -> ConeReference: ...

    async def get_cone(self, key: str) -> ConeReference: ...

    async def list_versions(
        self, *, tenant: str, task: str
    ) -> tuple[str, ...]: ...

    async def delete_cone(self, key: str, *, force: bool) -> None: ...

    def breach_history(
        self, *, limit: int, tenant: str | None,
        task: str | None, min_magnitude: float,
    ) -> AsyncIterator[dict[str, Any]]: ...
```

Six RPC admin : register, get, list, delete, breach_history (audit). Protocol-like pour injection.

6.2 Client haut-niveau avec cache TTL

```
class ConeRegistryClient:
    """High-level façade over an :class:`AdminTransport`.

    Adds:
    * input sanitation (canonical key formatting),
    * caching: recently fetched cones are cached with a short TTL to
      avoid hammering the control plane on repeated ``GetCone`` during
      bot planning.
    """

    def __init__(self, transport: AdminTransport, *,
                 cache_ttl_s: float = 30.0) -> None:
        self._transport = transport
        self._cache: dict[str, tuple[float, ConeReference]] = {}
        self._cache_ttl_s = cache_ttl_s

    async def get(self, key: str, *, use_cache: bool = True
                  ) -> ConeReference:
        if use_cache and key in self._cache:
            when, cached = self._cache[key]
            if self._now() - when < self._cache_ttl_s:
                return cached
        ref = await self._transport.get_cone(key)
        self._cache[key] = (self._now(), ref)
        return ref
```

Cache TTL 30 s pour ne pas marteler l'Admin service pendant la planification du bot.

6.3 Breach history : audit des dépassements

Chaque fois qu'un centroïde dépasse le rayon de projection, l'arbitre émet une entrée de **breach** : {cone_key, observed_delta, timestamp, request_id}. L'AdminService expose BreachHistory() en streaming pour analyse post-mortem et ajustement du cône.

> ***Doctrine opérationnelle** : un cône trop serré provoque des rejets massifs (MODE_COLLAPSE fréquent) ; un cône trop large laisse passer des réponses hors politique. On monitore donc la breach rate en continu (métrique Prometheus efe_swarm_breach_total) et on élargit / resserre les ancres par EvolveCone.*

7. L'arbitre et les critères de convergence

L'**arbitre** est l'algorithme central exécuté par le cœur Rust. À chaque round, il agrège l'état des nano-agents, décide de continuer ou de conclure, et applique les budgets. Son API Python côté plateforme est :

```
@dataclass(frozen=True, slots=True)
class ExecuteRequest:
    """Client-side view of the ``Execute`` RPC input."""
    spec: SwarmSpec
    raw_prompt: str = ""
    request_id: str | None = None
    decode_mode: DecodeMode = field(default_factory=DecodeMode)
    deadline_ms: int | None = None  # absolute deadline override


@dataclass(frozen=True, slots=True)
class ExecuteResponse:
    """Client-side view of the ``Execute`` RPC output."""
    swarm_id: str
    request_id: str
    payload: DecodedPayload
    confidence: float
    verdict: Verdict
    rounds_executed: int
    total_ms: float
    alignment_delta: float
```

La RPC Execute : une requête, une réponse. alignment_delta est la distance max centroïde→cône sur tout l'essaim.

7.1 Pseudo-code de la boucle d'arbitrage

```
# Pseudo-code (Rust en réalité)
state = init_swarm(spec)
budget = spec.budget
stability_buf = RingBuffer(spec.convergence.stability_window)

for t in range(spec.max_rounds):
    if elapsed_ms() > budget.max_wall_clock_ms: return CONTINUE
    if energy_wh() > budget.max_energy_wh: return CONTINUE
    if hbm_used() > budget.max_hbm_bytes: return CONTINUE

    # Chaque nano-agent produit un vecteur latent
    vectors = [a.step(state) for a in state.agents]

    mu_t   = weighted_mean(vectors)
    var_t  = variance(vectors)
    grad_t = mu_t - state.mu_prev

    # Projection sur le cône (rejet si hors-politique)
    delta = cone_distance(mu_t, state.cone)
    if delta > spec.max_projection_distance:
        return DIVERGENCE

    stability_buf.push(mu_t)
    stab_t = stddev_pairwise(stability_buf)

    if (var_t  < spec.convergence.epsilon_variance  and
        norm(grad_t) < spec.convergence.epsilon_gradient and
        stab_t < spec.convergence.epsilon_stability):
        return CONVERGED(mu_t, confidence_from(var_t, stab_t))

    if detect_mode_collapse(vectors):
        return MODE_COLLAPSE

    state = update_state(state, vectors, mu_t)

return CONTINUE  # max_rounds atteint
```

Boucle d'arbitrage : 3 critères de convergence + 3 budgets + 2 détections de pathologie.

7.2 Détection de mode collapse

Un **mode collapse** se produit quand la population tombe sur un unique vecteur trivial (souvent le zéro ou un biais résiduel). L'arbitre le détecte par une chute brutale de l'entropie spectrale des vecteurs latents sur 2 rounds consécutifs. Traitement : fire MODE_COLLAPSE, refuser la réponse, recommandation client : augmenter les agents na-crit qui forcent la diversité.

7.3 Confiance calibrée

Le confidence retourné par l'arbitre n'est pas une probabilité a posteriori brute — c'est une **calibration** sur la dispersion finale et la stabilité : une convergence qui a pris peu de rounds avec faible variance produit confidence ≈ 1,0 ; une convergence arrachée au dernier round avec variance proche du seuil produit confidence ≈ 0,5. Le client discrimine via DEFAULT_CONFIDENCE_FLOOR = 0,55.

8. Le service efe_orch — orchestrateur de jobs

efe_orch pilote les **jobs de fine-tuning** : il accepte une demande, choisit un worker, supervise la progression via heartbeats, relance les workers morts, et marque le job terminé. Il stocke son état en PostgreSQL et communique via gRPC avec les workers.

8.1 Modules clés

Module	Rôle
<code>efe_orch.dispatcher</code>	Sélection de worker (availability, load, tenant affinity)
<code>efe_orch.scheduler</code>	Priorisation des jobs en attente
<code>efe_orch.state_machine</code>	FSM des jobs : queued→running→completed|failed|cancelled
<code>efe_orch.heartbeat</code>	Détection de worker silencieux (timeout configurable)
<code>efe_orch.retry</code>	Politique de retry avec backoff exponentiel + jitter
<code>efe_orch.services</code>	Orchestration applicative de haut niveau
<code>efe_orch.observability</code>	Traces OTel + métriques Prometheus
<code>efe_orch.db</code>	Repositories SQL async (SQLAlchemy 2.0)
<code>efe_orch.api</code>	REST FastAPI : submit / cancel / status

8.2 FSM des jobs

```
# Transitions autorisées (whitelist) — CAS optimiste sur state_version
ALLOWED: dict[JobState, frozenset[JobState]] = {
    JobState.QUEUED:    frozenset({JobState.RUNNING, JobState.CANCELLED}),
    JobState.RUNNING:   frozenset({JobState.COMPLETED, JobState.FAILED,
                                   JobState.CANCELLED}),
    JobState.COMPLETED: frozenset(),
    JobState.FAILED:    frozenset({JobState.QUEUED}),  # retry
    JobState.CANCELLED: frozenset(),
}

def transition(from_: JobState, to: JobState) -> None:
    if to not in ALLOWED[from_]:
        raise IllegalTransition(f"{from_.name} → {to.name} forbidden")
```

FSM stricte : toute transition non whitelistée lève IllegalTransition. La version optimiste (CAS) protège des writes concurrents.

8.3 Retry avec backoff exponentiel + jitter

```
def compute_backoff_ms(
    attempt: int, *,
    base_ms: int = 1000,
    max_ms: int = 60_000,
    jitter_ratio: float = 0.25,
    rng: random.Random,
) -> int:
    """Exponential backoff with decorrelated jitter (AWS pattern)."""
    exp = base_ms * (2 ** (attempt - 1))
    capped = min(exp, max_ms)
    jitter = int(capped * jitter_ratio)
    return rng.randint(capped - jitter, capped + jitter)
```

Backoff exponentiel + jitter déterministe (rng seedé) pour reproductibilité des tests.

9. Le service efe_worker — trainer

efe_worker exécute les jobs de fine-tuning reçus de l'orchestrateur. Il supporte 5 méthodes d'entraînement et gère les snapshots de données, le transport vers S3, et l'observabilité du training.

9.1 Les 5 méthodes d'entraînement

Méthode	Fichier	Description
<code>lora</code>	methods/lora.py	PEFT LoRA — adapters low-rank (r ∈ [1, 64])
<code>classif</code>	methods/classif.py	Classification multiclasse avec cross-entropy
<code>infonce</code>	methods/infonce.py	Contrastive InfoNCE sur paires positives
<code>triplet</code>	methods/triplet.py	Triplet loss (anchor/positive/negative)
<code>distill</code>	methods/distill.py	Distillation enseignant→élève (KL sur logits)

9.2 Extrait : méthode LoRA

```
class LoRAMethod:
    name = "lora"

    def __init__(
        self, hyperparameters: dict[str, Any] | None = None,
    ) -> None:
        hp = dict(hyperparameters or {})
        self.r = int(hp.get("r", 8))
        if not (1 <= self.r <= 64):
            raise InvalidHyperparameters(
                f"LoRA r must be 1..64, got {self.r}")
        self.alpha = int(hp.get("alpha", 2 * self.r))
        self.lora_dropout = float(hp.get("lora_dropout", 0.05))
        tm = hp.get("target_modules", ("q_proj", "v_proj"))
        if isinstance(tm, str):
            if tm != "all-linear":
                raise InvalidHyperparameters(
                    f"target_modules string {tm!r} is not 'all-linear'")
            self.target_modules: Any = tm
        else:
            self.target_modules = tuple(tm)
            if not self.target_modules:
                raise InvalidHyperparameters(
                    "target_modules must contain ≥ 1 entry")
        self._label_map: dict[str, int] = {}
```

Extrait de efe_worker/methods/lora.py — validation stricte des hyperparamètres à la construction. Les défauts MVP (r=8, alpha=16, dropout=0.05) sont les valeurs sûres.

9.3 Fondation commune : Method protocol

```
class Model(Protocol):
    """Thin protocol over HF AutoModel that worker code needs."""
    def forward(self, batch: Batch) -> LossOutput: ...
    def save_pretrained(self, path: Path) -> None: ...
    def num_parameters(self) -> int: ...


@dataclass(frozen=True, slots=True)
class LossOutput:
    loss: torch.Tensor
    logits: torch.Tensor | None = None
    metrics: dict[str, float] = field(default_factory=dict)


class Method(Protocol):
    """Interface every training method must implement."""
    name: str
    def build_model(
        self, snapshot: Snapshot, *, base_model_id: str,
    ) -> Model: ...
    def build_sampler(self, snapshot: Snapshot) -> BaseSampler: ...
    def hyperparameter_summary(self) -> dict[str, Any]: ...
```

Contrat minimal : build_model + build_sampler + hyperparameter_summary. Tout le reste est dans la base commune.

9.4 Détection de divergence en training

```
# Dans la boucle d'entraînement
def train_step(batch, model, optim) -> float:
    out = model.forward(batch)
    if not torch.isfinite(out.loss):
        raise TrainingDiverged(
            f"non-finite loss at step {step}: {out.loss.item()}")
    out.loss.backward()
    grad_norm = torch.nn.utils.clip_grad_norm_(
        model.parameters(), max_norm=1.0)
    if grad_norm > GRAD_EXPLOSION_THRESHOLD:
        raise TrainingDiverged(
            f"grad norm {grad_norm:.1f} > {GRAD_EXPLOSION_THRESHOLD}")
    optim.step()
    optim.zero_grad(set_to_none=True)
    return float(out.loss.item())
```

Sentinelles : loss non-finite + grad-norm explosion → TrainingDiverged, remontée au orch qui marque le job failed sans retry.

10. Le service efe_registry — artefacts + signature

Après un job réussi, l'artefact (poids LoRA + manifeste + metrics) est publié dans le **registry**. Celui-ci le signe, le stocke sur un bucket compatible OCI (S3 / Artifact Registry), et vérifie la signature à chaque chargement aval.

10.1 Trois méthodes de signature supportées

```
class SigningMethod(str, Enum):
    KEY_KMS = "key_kms"              # KMS GCP/AWS managed key
    KEYLESS_OIDC = "keyless_oidc"    # Sigstore fulcio + OIDC
    ATTESTATION = "attestation"      # SLSA attestation in-toto


class VerificationLayer(str, Enum):
    L1_PRE_PUBLISH = "L1"   # à la publication
    L2_ADMISSION = "L2"     # par l'admission controller
    L3_PRE_LOAD = "L3"      # juste avant chargement en mémoire
    L4_RUNTIME = "L4"       # probes périodiques en prod


@dataclass(frozen=True, slots=True)
class VerificationInput:
    """What the verifier is asked to verify."""
    artifact_id: str
    layer: VerificationLayer
    oci_ref: str | None = None
    blob_path: Path | None = None
    signature_path: Path | None = None
    certificate_path: Path | None = None
    require_attestations: tuple[str, ...] = field(default_factory=tuple)


@dataclass(frozen=True, slots=True)
class VerificationResult:
    ok: bool
    method: SigningMethod
    key_id: str | None
    rekor_uuid: str | None      # ID transparency log (Sigstore)
    verified_at_ns: int
    layer: VerificationLayer
    details: dict[str, str]
```

3 méthodes (KMS, keyless, SLSA) × 4 couches de vérification = 12 points de contrôle possibles. Défaut prod : L1 + L3 obligatoires.

10.2 Modules du registry

Module	Rôle
<code>efe_registry.core</code>	Entités Artefact / Manifest / Digest
<code>efe_registry.admission</code>	Admission controller : politique qui peut publier quoi
<code>efe_registry.signing</code>	Signer (publish) + Verifier (consume) + cache TTL
<code>efe_registry.canary</code>	Publication canari : artefact visible uniquement à X % du trafic
<code>efe_registry.shadow</code>	Mode shadow : artefact évalué en miroir, jamais servi
<code>efe_registry.router</code>	Résolution (tenant, task, version) → oci_ref
<code>efe_registry.serving</code>	Streaming de poids vers efe_inference avec vérif L3
<code>efe_registry.rollback</code>	Bascule en état stable sur incident aval
<code>efe_registry.nats</code>	Émission d'événements de cycle de vie

10.3 Verification cache

```
# efe_registry/signing/cache.py — cache TTL de résultats de vérif
# Évite de re-vérifier le même artefact à chaque requête inference.

@dataclass(slots=True)
class _CacheEntry:
    verified_at_ns: int
    result: VerificationResult


class VerificationCache:
    def __init__(self, *, ttl_s: float = 300.0, max_entries: int = 10_000):
        self._ttl_ns = int(ttl_s * 1e9)
        self._max = max_entries
        self._cache: dict[str, _CacheEntry] = {}

    def get(self, artifact_id: str) -> VerificationResult | None:
        entry = self._cache.get(artifact_id)
        if entry is None:
            return None
        if time.monotonic_ns() - entry.verified_at_ns > self._ttl_ns:
            self._cache.pop(artifact_id, None)
            return None
        return entry.result

    def put(self, artifact_id: str, r: VerificationResult) -> None:
        if len(self._cache) >= self._max:
            # Drop oldest (insertion order thanks to dict ordering 3.7+)
            first = next(iter(self._cache))
            self._cache.pop(first, None)
        self._cache[artifact_id] = _CacheEntry(time.monotonic_ns(), r)
```

Cache TTL 5 min avec éviction FIFO pour borner l'usage mémoire sous pression.

11. Le service efe_gate — comparateur + promotion

efe_gate décide si un **candidat** fraîchement entraîné peut remplacer le **champion** en production. Il compare leurs métriques par des tests statistiques rigoureux et produit un verdict par métrique + un verdict global.

11.1 Verdicts par métrique

```
class MetricComparisonVerdict(str, Enum):
    PASS = "pass"
    FAIL = "fail"
    AMBIGUOUS = "ambiguous"
    BETTER = "better"
    BUDGET_EXCEEDED = "budget_exceeded"
    UNCOMPARABLE = "uncomparable"   # missing from one side


@dataclass(frozen=True, slots=True)
class MetricComparison:
    metric_name: str
    role: MetricRole
    verdict: MetricComparisonVerdict
    champion_value: float | None
    candidate_value: float
    delta: float | None
    ci_lower: float | None
    ci_upper: float | None
    test_name: TestName
    p_value: float | None
    details: dict[str, Any] = field(default_factory=dict)
```

Extrait de efe_gate/comparator/engine.py — chaque comparaison produit un verdict + CI + p-value + nom du test utilisé.

11.2 Tests statistiques supportés

Nom du test	Usage typique	Statistique
<code>welch_test</code>	Comparaison de moyennes inégales	t de Welch
<code>mcnemar_test</code>	Paires binaires (success/fail)	χ^2 McNemar
<code>tost_equivalence</code>	Test d'équivalence à un seuil δ	Two One-Sided Tests
<code>bootstrap_delta_ci</code>	Choisir une différence non-paramétrique	Bootstrap percentile 10■ tirages

11.3 Pipeline de décision

```
# Simplification du flux efe_gate.decision
def decide_promotion(
    bundle_champ: MetricsBundle,
    bundle_cand: MetricsBundle,
    policy: GatePolicy,
) -> GateDecision:
    # 1) Comparaison par métrique (engine pur)
    comparisons = ComparisonEngine().compare(bundle_champ, bundle_cand)

    # 2) Budget: coût, latence, énergie
    for c in comparisons:
        if c.role == MetricRole.BUDGET and c.verdict == "budget_exceeded":
            return GateDecision.REJECT(reason="budget_exceeded",
                                       details={"metric": c.metric_name})

    # 3) Métrique obligatoire qui échoue → reject
    for c in comparisons:
        if c.role == MetricRole.MUST_NOT_REGRESS and c.verdict == "fail":
            return GateDecision.REJECT(reason="hard_regression",
                                       details={"metric": c.metric_name})

    # 4) Équivalence sur métriques secondaires → pass même sans progrès
    equiv = all(c.verdict in ("pass", "better")
                for c in comparisons if c.role == MetricRole.SECONDARY)

    # 5) Au moins une métrique primaire doit être BETTER
    primary_better = any(c.verdict == "better"
                         for c in comparisons
                         if c.role == MetricRole.PRIMARY)

    if equiv and primary_better:
        return GateDecision.PROMOTE(comparisons=comparisons)
    return GateDecision.HOLD(comparisons=comparisons)
```

La logique de décision : budgets durs, non-régression obligatoire, équivalence secondaire, amélioration primaire. Sans progrès clair → HOLD.

11.4 Rollback réactif

Le sous-paquet efe_gate.rollback s'exécute **après** une promotion. Si la métrique live diverge significativement des prédictions de la campagne de candidat, il peut déclencher un retour au champion précédent. Couplé à efe_rollout, c'est le filet de sécurité de dernier recours.

12. Le service efe_rollout — canari + bandits

efe_rollout orchestre le déploiement progressif. Deux stratégies sont implémentées : **étapes discrètes** (canary 1 %→5 %→25 %→100 %) et **bandits multi-armés** pour les A/B tests avec plus de deux variantes.

12.1 Trois samplers de bandit

```
class BanditSampler(Protocol):
    def choose(
        self, arms: Sequence[BanditArmState],
    ) -> BanditArmState: ...


class ThompsonSampler:
    """Thompson-sampling over Beta posteriors."""

    __slots__ = ("_minimum_allocation_bps", "_rng")

    def __init__(
        self, *, seed: int = 20260420,
        minimum_allocation_pct: int = 0,
    ) -> None:
        if not 0 <= minimum_allocation_pct <= 50:
            raise ValueError("minimum_allocation_pct must be in [0, 50]")
        self._rng = random.Random(seed)
        self._minimum_allocation_bps = minimum_allocation_pct * 100

    def choose(
        self, arms: Sequence[BanditArmState],
    ) -> BanditArmState:
        if not arms:
            raise BanditError("thompson sampler requires at least one arm")
        # Minimum-allocation guard (anti-starvation)
        if self._minimum_allocation_bps > 0:
            n_arms = len(arms)
            roll = self._rng.randint(0, 9_999)
            if roll < self._minimum_allocation_bps * n_arms:
                return arms[roll % n_arms]
        # Standard Thompson sampling — pick argmax(Beta sample)
        best = arms[0]
        best_score = self._rng.betavariate(best.alpha, best.beta)
        for a in arms[1:]:
            s = self._rng.betavariate(a.alpha, a.beta)
            if s > best_score:
                best, best_score = a, s
        return best
```

Thompson sampling sur posteriors Beta(α, β). Seedé pour reproductibilité. Garde anti-starvation via min-allocation.

12.2 UCB1 et ε-greedy

```
class Ucb1Sampler:
    """Deterministic UCB1 : mean + sqrt(2 ln T / n)."""

    def choose(self, arms: Sequence[BanditArmState]) -> BanditArmState:
        total_n = sum(a.trials for a in arms)
        # Bras non encore tirés → priorité absolue
        untried = [a for a in arms if a.trials == 0]
        if untried:
            return untried[0]
        # UCB score
        ln_t = math.log(max(1, total_n))
        best = arms[0]
        best_score = best.reward_mean + math.sqrt(2 * ln_t / best.trials)
        for a in arms[1:]:
            s = a.reward_mean + math.sqrt(2 * ln_t / a.trials)
            if s > best_score:
                best, best_score = a, s
        return best


class EpsilonGreedySampler:
    """ε decays geometrically each step."""

    def __init__(self, *, epsilon: float, decay: float, seed: int):
        self._eps = epsilon
        self._decay = decay
        self._rng = random.Random(seed)

    def choose(self, arms: Sequence[BanditArmState]) -> BanditArmState:
        if self._rng.random() < self._eps:
            picked = self._rng.choice(list(arms))
        else:
            picked = max(arms, key=lambda a: a.reward_mean)
        self._eps *= self._decay
        return picked
```

UCB1 : déterministe, simple, mais doit balayer tous les bras une fois. ε-greedy : décroissance géométrique d'exploration.

12.3 Layers d'expérimentation

Le sous-paquet efe_rollout.layers permet d'empiler plusieurs expérimentations orthogonales sur le même trafic — inspiré du *Layered Experimentation Framework* de Google. Chaque layer consomme une fraction du hash utilisateur (par ex. bits 0-3 → layer A, bits 4-7 → layer B). Cela garantit l'indépendance statistique entre expérimentations concurrentes.

13. Le service efe_inference — backends + streaming

efe_inference est la surface d'exécution en production. Il est volontairement **agnostique au modèle** : un registre de backends pluggables le connecte à Claude, OpenAI, Mistral, vLLM ou à des adaptateurs PEFT locaux.

13.1 Protocole BackendAdapter

```
@dataclass(frozen=True, slots=True)
class BackendRequest:
    """The input to any BackendAdapter."""
    request_id: str
    tenant_id: str
    task_id: str
    model: str
    textual: str | None
    messages: tuple[dict[str, Any], ...] | None
    sampling: SamplingParams
    tools: tuple[ToolDef, ...]
    tool_choice: str
    stream: bool
    timeout_s: float


@dataclass(frozen=True, slots=True)
class BackendResponse:
    """Backend-produced response, pre-safety-postflight."""
    text: str
    tool_calls: tuple[dict[str, Any], ...] = field(default_factory=tuple)
    finish_reason: FinishReason = FinishReason.STOP
    usage: Usage = field(default_factory=lambda: Usage(
        prompt_tokens=0, completion_tokens=0, total_tokens=0))
    raw: dict[str, Any] = field(default_factory=dict)
    backend_latency_ms: float = 0.0


class BackendAdapter(Protocol):
    name: str
    kind: BackendKind

    async def generate(
        self, req: BackendRequest,
    ) -> BackendResponse: ...

    async def stream(
        self, req: BackendRequest,
    ) -> AsyncIterator[ChatChunk]: ...

    async def health(self) -> HealthStatus: ...
```

Extrait de efe_inference/backends/protocol.py — 5 backends implémentent ce contrat.

13.2 Backends implémentés

Backend	Kind	Usage
<code>claude</code>	frontier	Anthropic Claude via SDK
<code>openai</code>	frontier	OpenAI GPT via SDK
<code>mistral</code>	frontier	Mistral hosted / local
<code>vllm</code>	self_hosted	Serveur vLLM interne
<code>peft</code>	self_hosted	Adapter LoRA chargé via efe_registry
<code>mock</code>	test	Déterministe, offline, pour tests

13.3 Sélecteur et résilience

- **selector.py** — choisit un backend en fonction du task, du tenant et de la politique de rollout active.
- **resilience/** — circuit breaker par backend + timeout + retry avec backoff.
- **cache/** — cache de réponses identiques (hash du prompt + paramètres) avec TTL par tenant.
- **streaming/** — SSE (Server-Sent Events) via sse-starlette.
- **safety/** — post-flight checks : filtrage PII, limite de longueur, détection de patterns indésirables.
- **templating/** — rendu Jinja de prompts avec sandboxing.
- **tools/** — registre de tools (function calling).
- **accounting/** — comptabilisation tokens + coût par tenant.

13.4 Observabilité intégrée

Chaque generate() émet trois signaux : une **trace OTel** (span avec attributs tenant/task/model), une **métrique Prometheus** (efe_inference_duration_seconds) et un **événement NATS** sur efe.inference.events que efe_drift consomme pour la détection de dérive.

14. Le service efe_drift — détection multi-méthode

efe_drift observe la production en continu et détecte tout **décalage de distribution** entre les features actuelles et une baseline scellée. Sept familles de détecteurs sont implémentées en **Python pur** (pas de SciPy) pour la portabilité et la reproductibilité bit-pour-bit.

14.1 Les détecteurs

Fichier	Méthode	Orientation
<code>psi.py</code>	Population Stability Index	higher_is_worse
<code>psi.py</code> (compagnon)	Jensen-Shannon divergence	higher_is_worse
<code>ks.py</code>	Kolmogorov-Smirnov (2-échantillons)	higher_is_worse
<code>chi_squared.py</code>	χ^2 sur table de contingence, sévérité sur p-value	lower_is_worse
<code>wasserstein.py</code>	Wasserstein-1 (Earth Mover)	higher_is_worse
<code>mmd.py</code>	Maximum Mean Discrepancy (RBF)	higher_is_worse
<code>classifier.py</code>	AUC d'un classifieur baseline-vs-current (Rabanser 2019)	higher_is_worse
<code>sequential.py</code>	CUSUM / Page séquentiel	higher_is_worse

14.2 Formule et extrait PSI

Formule (Siddiqi 2005) : $PSI = \Sigma\ (p_i - q_i) \cdot \ln(p_i / q_i)$

```
def compute_psi(
    current_counts: Sequence[int],
    baseline_counts: Sequence[int],
    *,
    eps: float = 1e-6,
) -> float:
    """Return the PSI between two aligned histograms.

    Raises :class:`HistogramBinMismatch` if bin counts differ.
    Returns ``NaN`` if either total is zero.
    """
    if len(current_counts) != len(baseline_counts):
        raise HistogramBinMismatch(
            f"PSI: histograms have {len(current_counts)} vs "
            f"{len(baseline_counts)} bins")
    n_cur = sum(current_counts)
    n_base = sum(baseline_counts)
    if n_cur == 0 or n_base == 0:
        return float("nan")
    psi = 0.0
    for c, b in zip(current_counts, baseline_counts, strict=True):
        p = max(eps, c / n_cur)
        q = max(eps, b / n_base)
        psi += (p - q) * math.log(p / q)
    return psi
```

Extrait de efe_drift/detection/psi.py — pure-Python, lissage ε=1e-6 contre log(0), vérification stricte des bin counts.

14.3 Classification de sévérité (monotone)

```
@dataclass(frozen=True, slots=True)
class SeverityThresholds:
    warning: float | None = None
    major: float | None = None
    critical: float | None = None
    orientation: Orientation = "higher_is_worse"

    def __post_init__(self) -> None:
        # Valide la monotonicité — protège contre une inversion accidentelle
        vals = [(n, v) for n, v in (
            ("warning", self.warning), ("major", self.major),
            ("critical", self.critical)) if v is not None]
        if self.orientation == "higher_is_worse":
            for (_, a), (_, b) in pairwise(vals):
                if a > b:
                    raise ValueError("thresholds must be non-decreasing")
        else:
            for (_, a), (_, b) in pairwise(vals):
                if a < b:
                    raise ValueError("thresholds must be non-increasing")

    def classify(self, score: float) -> Severity:
        if self.orientation == "higher_is_worse":
            if self.critical is not None and score >= self.critical:
                return "critical"
            if self.major is not None and score >= self.major:
                return "major"
            if self.warning is not None and score >= self.warning:
                return "warning"
            return "none"
        # lower_is_worse (p-values)
        if self.critical is not None and score <= self.critical:
            return "critical"
        if self.major is not None and score <= self.major:
            return "major"
        if self.warning is not None and score <= self.warning:
            return "warning"
        return "none"


DEFAULT_PSI = SeverityThresholds(warning=0.10, major=0.25, critical=0.40)
DEFAULT_CHI_SQUARED_P = SeverityThresholds(
    warning=0.05, major=0.01, critical=0.001,
    orientation="lower_is_worse")
```

Invariant I3 (monotonicité) validé à la construction. Deux orientations : higher_is_worse (PSI) ou lower_is_worse (p-values).

14.4 Schemas API exposés

```
class DriftResultSchema(BaseModel):
    model_config = ConfigDict(extra="forbid")
    evaluation_id: str
    tenant_id: str
    task_id: str
    feature_id: str
    feature_name: str
    baseline_id: str
    method: str
    score: float
    severity: str
    computed_at: datetime
    window_start: datetime
    window_end: datetime
    p_value: float | None = None
    multi_scale_window: str | None = None
    details: dict[str, Any] = Field(default_factory=dict)
    sample_count_current: int
    sample_count_baseline: int


class ThresholdsSchema(BaseModel):
    model_config = ConfigDict(extra="forbid")
    warning: float | None = None
    major: float | None = None
    critical: float | None = None
    orientation: Literal["higher_is_worse", "lower_is_worse"] = "higher_is_worse"


class AlertRuleCreateRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    tenant_id: str
    method: str
    thresholds: ThresholdsSchema
    channels: list[ChannelRefSchema] = Field(min_length=1)
    task_id: str | None = None
    feature_id: str | None = None
    window_kind: Literal["1h", "1d", "7d"] | None = None
    consecutive_breaches: int = Field(default=3, ge=1, le=20)
    suppression_s: int = Field(default=1800, ge=0, le=86_400)
    name: str | None = None
    notes: str | None = None
```

Extrait de efe_drift/api/schemas.py — Pydantic v2 strict (extra='forbid') avec contraintes métier (min_length, ge, le).

14.5 Canal d'alerting NATS

```
@dataclass(frozen=True, slots=True)
class NatsAlertChannel:
    publisher: Publisher
    subject: str = "efe.drift.alert"
    clock: Clock = field(default_factory=SystemClock)
    _name: str = "nats"

    @property
    def name(self) -> str:
        return self._name

    async def dispatch(self, payload: IncidentPayload) -> DispatchResult:
        try:
            await self.publisher.publish(self.subject, payload.to_dict())
        except Exception as exc:
            return DispatchResult(
                ok=False, channel=self._name,
                error=f"{type(exc).__name__}: {exc}")
        return DispatchResult(
            ok=True, channel=self._name,
            delivered_at=self.clock.now(),
            extra={"subject": self.subject})
```

Extrait de efe_drift/alerting/channels/nats.py — publication sur efe.drift.alert. Tout échec est capturé et retourné (jamais propagé).

15. Le service efe_platform — bot + swarm client

efe_platform est le **SDK public** et le **bot** qui compose des requêtes swarm. Il expose une API concise : l'utilisateur appelle bot.answer("...", tenant="...") et obtient un BotReply complet avec trace.

15.1 Composition du bot

```
class Bot:
    """Top-level orchestrator exposing answer()."""

    def __init__(
        self, *,
        swarm_client: SwarmClient,
        cone_registry: ConeRegistryClient,
        llm_provider: LlmProvider,
        tool_registry: ToolRegistry,
        memory: ConversationMemory,
        budget_ledger: BudgetLedger,
        policy: PolicyEngine,
        planner: Planner,
        audit_writer: AuditWriter | None = None,
    ) -> None:
        self._swarm = swarm_client
        self._cones = cone_registry
        self._llm = llm_provider
        self._tools = tool_registry
        self._memory = memory
        self._budget = budget_ledger
        self._policy = policy
        self._planner = planner
        self._audit = audit_writer

    async def answer(
        self, prompt: str, *, tenant: str, task: str = "default",
        budget: Budget | None = None,
    ) -> BotReply:
        self._policy.enforce_preflight(prompt=prompt, tenant=tenant)
        if budget is None:
            budget = self._budget.default_budget(tenant)
        with self._budget.hold(budget, tenant=tenant):
            plan = await self._planner.plan(prompt, tenant=tenant, task=task)
            return await self._execute_plan(plan, prompt=prompt,
                                            tenant=tenant, task=task)
```

Extrait simplifié de efe_platform/bot/agent.py — le bot orchestre 9 sous-systèmes, tous injectés.

15.2 Trace Event : observabilité par défaut

```
@dataclass(frozen=True, slots=True)
class TraceEvent:
    """One entry in a BotReply.trace."""
    kind: str
    summary: str
    ts_ms: int
    payload: Mapping[str, Any] = field(default_factory=dict)

    def to_dict(self) -> dict[str, Any]:
        return {
            "kind": self.kind,
            "summary": self.summary,
            "ts_ms": self.ts_ms,
            "payload": dict(self.payload),
        }
```

Chaque décision du bot est journalisée dans la trace. Types : plan_step, tool_call, swarm_execute, policy_check, budget_hold, etc.

15.3 Hiérarchie d'erreurs

```
class EfePlatformError(Exception):
    """Root of the platform-specific error hierarchy."""
    retryable: bool = False
    code: str = "efe_platform_error"

    def __init__(self, message: str, *, detail: Any = None) -> None:
        super().__init__(message)
        self.message = message
        self.detail = detail

    def to_dict(self) -> dict[str, Any]:
        payload: dict[str, Any] = {
            "code": self.code, "message": self.message,
            "retryable": self.retryable,
        }
        if self.detail is not None:
            payload["detail"] = self.detail
        return payload


class ConfigError(EfePlatformError):
    code = "config_error"
    retryable = False


class LlmProviderUnavailable(ConfigError):
    """No LLM provider backing is wired up — refuse to start the bot."""

class SwarmUnavailable(EfePlatformError):
    code = "swarm_unavailable"
    retryable = True

class BudgetExceeded(EfePlatformError):
    code = "budget_exceeded"
    retryable = False
```

Racine unique + attribut retryable consommé par la boucle de retry + code snake_case stable pour logs.

15.4 Catalogue typé

```
@dataclass(frozen=True, slots=True)
class CatalogTask:
    """A single named task exposed by the catalog."""
    id: str
    description: str
    tenant: str
    cone_key: str
    dim_latent: int
    anchors: tuple[str, ...] = ()
    tags: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        if not self.id:
            raise ValueError("task.id must be non-empty")
        if not self.tenant:
            raise ValueError("task.tenant must be non-empty")
        if not self.cone_key.startswith(f"{self.tenant}/"):
            raise ValueError(
                f"cone_key {self.cone_key!r} must start with tenant/...")
        if self.dim_latent <= 0:
            raise ValueError("task.dim_latent must be > 0")
```

Extrait de efe_platform/catalog/models.py — le catalogue énumère les tasks exposées par le bot. Invariant : cone_key préfixé du tenant.

16. Sécurité : HMAC, cônes, signing keyless, RBAC

16.1 Huit domaines de sceaux HMAC

Domain tag	Usage
<code>efe-drift/baseline/v1</code>	Scellement d'une baseline de feature
<code>efe-drift/nats/v1</code>	Signature des messages NATS sortants
<code>efe-drift/webhook/v1</code>	Signature des payloads webhook (X-EFE-Signature)
<code>efe-drift/user-id/v1</code>	Hash stable des user-id pour traffic split déterministe
<code>efe-registry/artifact/v1</code>	Sceau du manifest d'un artefact (complément OIDC)
<code>efe-gate/decision/v1</code>	Sceau de décision de gate (audit)
<code>efe-orch/job/v1</code>	Sceau de commande de job envoyée à un worker
<code>efe-platform/catalog/v1</code>	Sceau du catalogue versionné

16.2 Fonction de sceau canonique

```
def canonical_json(obj: Any) -> bytes:
    """Stable canonical JSON encoding (I1)."""
    return json.dumps(
        obj, sort_keys=True, separators=(",", ":"),
        ensure_ascii=False,
    ).encode("utf-8")


def hmac_seal(key: bytes, obj: Any, tag: bytes) -> bytes:
    """HMAC-SHA256 with domain separation tag (I8)."""
    msg = tag + b"\x00" + canonical_json(obj)
    return hmac.new(key, msg, hashlib.sha256).digest()


def hmac_verify(key: bytes, obj: Any, tag: bytes, seal: bytes) -> bool:
    """Constant-time verification."""
    expected = hmac_seal(key, obj, tag)
    return hmac.compare_digest(expected, seal)


# Convenience constants
DOMAIN_TAG_BASELINE: bytes = b"efe-drift/baseline/v1\x00"
DOMAIN_TAG_NATS:     bytes = b"efe-drift/nats/v1\x00"
DOMAIN_TAG_WEBHOOK:  bytes = b"efe-drift/webhook/v1\x00"
DOMAIN_TAG_USER_ID:  bytes = b"efe-drift/user-id/v1\x00"
```

canonical_json assure l'unicité de l'empreinte. Le domain tag prévient toute confusion cross-canal. hmac.compare_digest est constant-time.

16.3 Signing keyless OIDC (Sigstore)

Pour les artefacts d'entraînement, EFE supporte le signing **keyless** basé sur Sigstore :

- Le worker obtient un jeton OIDC (GitHub Actions / Workload Identity).
- Il demande un certificat éphémère à **Fulcio** (CA Sigstore).
- Il signe le digest de l'artefact avec la clé privée éphémère.
- Il publie la signature + certificat dans **Rekor** (transparency log).

• Au chargement, le verifier extrait le rekor_uuid, re-vérifie contre Rekor, re-valide le certificat contre Fulcio, et confirme le sujet (tenant + repo autorisé).

16.4 RBAC et multi-tenancy

• Trois rôles : admin, operator, reader.

• JWT courts (1 h) signés par un IdP interne ; claim tenant_id enforce tenant isolation.

• RLS PostgreSQL : chaque table a une policy tenant_id = current_setting('app.tenant_id').

• Les cônes sont scopés par tenant/task : aucun cross-tenant read possible.

• Les quotas (rps, tokens/mois) sont token-bucket par tenant (efe_gate ou efe_inference).

17. Observabilité : OpenTelemetry, structlog, Prometheus

17.1 Trois piliers d'observabilité

Pilier	Stack	Usage dans EFE
Logs	structlog + stdout JSON	structlog.contextvars propage request_id/tenant/trace_id
Traces	OpenTelemetry + OTLP → Jaeger	Chaque RPC gRPC + chaque méthode critique est un span
Métriques	Prometheus client + /metrics	Compteurs + histogrammes par service (latency, errors, rps)

17.2 Exemple : propagation structlog

```
import structlog

log = structlog.get_logger(__name__)

async def handle_request(req: InferenceRequest) -> InferenceResponse:
    # Bind tenant & request_id sur tout le span
    with structlog.contextvars.bound_contextvars(
        request_id=req.request_id,
        tenant_id=req.tenant_id,
        task_id=req.task_id,
    ):
        log.info("inference.request.received", model=req.model)
        try:
            resp = await backend.generate(req)
        except BackendError as e:
            log.error("inference.backend.failed",
                      error_code=e.code, retryable=e.retryable)
            raise
        log.info("inference.request.served",
                 tokens=resp.usage.total_tokens,
                 latency_ms=resp.backend_latency_ms)
        return resp
```

bound_contextvars propage les clés sur tous les logs émis depuis la même tâche asyncio.

17.3 Métriques Prometheus standard

```
from prometheus_client import Counter, Histogram

INFERENCE_TOTAL = Counter(
    "efe_inference_total",
    "Total inference requests served",
    ["tenant_id", "task_id", "backend", "status"],
)

INFERENCE_DURATION = Histogram(
    "efe_inference_duration_seconds",
    "Inference request duration",
    ["tenant_id", "backend"],
    buckets=(0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0),
)

SWARM_VERDICT = Counter(
    "efe_swarm_verdict_total",
    "Swarm verdicts emitted",
    ["tenant", "task", "verdict"],
)

DRIFT_INCIDENT = Counter(
    "efe_drift_incident_total",
    "Drift incidents fired",
    ["tenant_id", "method", "severity"],
)
```

Pattern : un Counter par événement discret, un Histogram par durée. Labels minimaux pour éviter la cardinalité explosive.

17.4 SLO MVP

Service	SLI	SLO MVP	Budget mensuel
<code>efe_inference</code>	Disponibilité 2xx	99,5 %	3h 39m
<code>efe_inference</code>	Latence p95	< 600 ms	≤ 5 % des requêtes hors SLO
<code>efe_drift</code>	Détection → alerte	< 60 s	—
<code>efe_orch</code>	Démarrage job	< 15 s p95	—
<code>efe_registry</code>	Verify L3	< 200 ms p99	—

18. Cycle de vie complet d'une requête (trace)

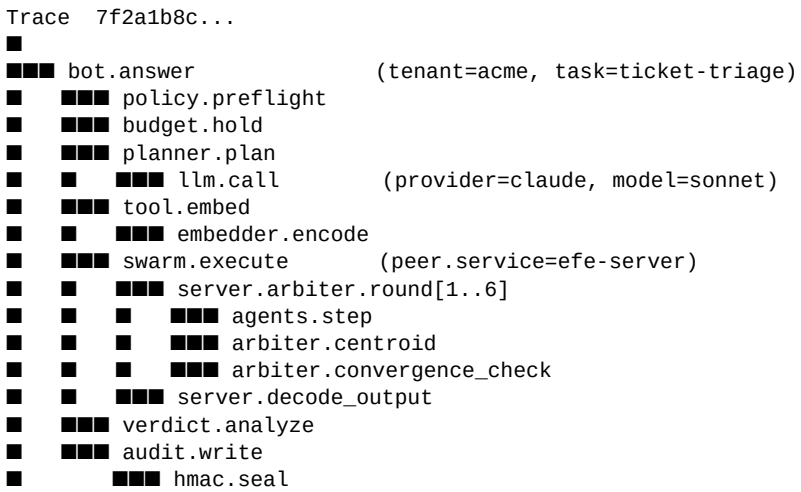
Suivons pas à pas ce qui se passe quand un utilisateur envoie "Classe ce ticket : ..." sur l'API publique du bot.

18.1 Schéma de la requête



14 étapes, ~400-600 ms p95 en MVP. L'étape [14] est asynchrone (fire-and-forget).

18.2 Instrumentation OTel (span graph)



Arbre OTel typique. Chaque niveau ajoute des attributs (e.g. round_id, centroid_dim, loss). Exporté via OTLP vers Jaeger.

18.3 Invariants garantis bout en bout

• **Idempotence** — le même request_id rejoué produit la même réponse (cache + event_id dédup).

• **Traçabilité** — tout événement journalisé porte trace_id OTel + request_id métier.

• **Intégrité** — la trace d'audit est scellée HMAC avec le DOMAIN_TAG_AUDIT avant stockage.

• **Multi-tenancy** — aucun accès cross-tenant n'est possible même en cas de bug applicatif (RLS + cônes).

• **Budgets durs** — tout dépassement (wall-clock, énergie, HBM) provoque une terminaison propre avec verdict CONTINUE.

19. Déploiement, empaquetage et outillage

19.1 Commandes just

```
# Extrait du README — recettes principales
just bootstrap     # installe rustup, uv, pre-commit (5-10 min)
just build         # build Rust + Python
just build-cuda    # kernels CUDA (requiert GPU)
just test          # fast + unit tests
just test-smoke    # vérif rapide (<10 s)
just bench         # benchmarks
just fmt           # formate tout (rustfmt + ruff format)
just lint          # ruff + clippy (fail si warning)
just clean         # purge les artefacts
just docker-up     # démarre NATS / Postgres / OTel / Jaeger
just docs-serve    # mkdocs serve sur :8000
```

Les recettes just unifient Rust + Python derrière une surface unique. Toute PR doit passer `just fmt && just lint && just test`.

19.2 Structure du repository

```
efe/
■■■ crates/              # Rust workspace
■   ■■■ efe-core/        # types, erreurs, orchestration
■   ■■■ efe-kernels/     # bindings CUDA + kernels
■   ■■■ efe-ffi/         # bindings PyO3
■   ■■■ efe-server/      # serveur gRPC
■■■ platform/
■   ■■■ pyproject.toml   # 9 paquets Python installables
■   ■■■ src/
■       ■■■ efe_platform/    (SDK + bot + swarm)
■       ■■■ efe_orch/        (orchestrateur jobs)
■       ■■■ efe_worker/      (trainer)
■       ■■■ efe_registry/    (artefacts + signing)
■       ■■■ efe_gate/        (promotion gates)
■       ■■■ efe_rollout/     (canari + bandits)
■       ■■■ efe_inference/   (serving)
■       ■■■ efe_drift/       (observabilité dérive)
■       ■■■ efe_ft/          (labeling + embedder)
■■■ protos/              # définitions gRPC .proto
■■■ configs/             # configs par environnement
■■■ docker/              # images dev + prod
■■■ docs/                # mkdocs (docs.efe.ai)
■■■ tests/               # tests d'intégration
■■■ scripts/             # hooks, release
■■■ justfile
■■■ rust-toolchain.toml
■■■ README.md
```

Le monorepo : crates Rust à la racine, 9 paquets Python sous platform/src/, le tout piloté par justfile.

19.3 Versions pinned (critique)

- **Rust 1.85** — épinglé par rust-toolchain.toml, aucune variation autorisée en CI.
- **Python 3.12** — géré par uv, caches de wheels partagés entre services.
- **CUDA 12.4** — optionnel pour développement CPU-only ; obligatoire pour just build-cuda et la prod GPU.
- **Postgres 16** — minimum pour les fonctions RLS et jsonb_path_query utilisées.
- **NATS 2.10** — JetStream obligatoire pour les streams durables.

19.4 Déploiement Kubernetes

- Chaque service est packagé en image Docker multi-stage (builder + runtime minimal *distroless*).
- Un Helm chart par service avec values.yaml par env.
- HPA sur CPU + métriques custom (efe_inference_queue_depth).
- Network policies : efe_inference → efe_registry et NATS, rien d'autre.
- Secrets via External Secrets Operator + AWS Secrets Manager (ou Vault).

20. Critères d'acceptation et gates techniques

20.1 Gates CI obligatoires

Gate	Outil	Critère
Format	<code>ruff format --check</code>	100 % propre
Lint Python	<code>ruff check</code>	Aucune erreur avec règles E/F/W/I/B/UP/SIM/N/ANN/RUF
Lint Rust	<code>cargo clippy -- -D warnings</code>	Aucun warning
Typage	<code>mypy --strict</code>	Aucune erreur, aucun <code>type: ignore</code> sans motif
Tests unit	<code>pytest tests/</code>	100 % verts
Tests service	<code>pytest tests_{svc}/</code>	100 % verts par service
Couverture	<code>pytest --cov</code>	≥ 90 % global, ≥ 95 % efe_platform/swarm
Build Rust	<code>cargo build --all-features</code>	Succès
Smoke e2e	<code>just test-smoke</code>	< 10 s, couvre les 9 services

20.2 Objectifs de performance MVP

Capacité	Cible MVP
Latence swarm.execute p95	< 600 ms
Latence swarm.execute p99	< 1 200 ms
Ingestion d'événements / pod	≥ 5 000 evt/s
Débit inference / pod (peft)	≥ 200 rps sur GPU T4
Détection dérive sur 10 k events	< 30 s
Rollback rollout	< 30 s après signal
Verify artefact L3	< 200 ms p99

20.3 Critères de sécurité MVP

- Tous les usages HMAC disposent d'un domain tag dédié (I8).
- Les artefacts sont signés L1 + vérifiés L3 (obligatoire).
- PII linter actif sur toute ingestion avec NFKC + depth-limit.
- JWT ≤ 1 h, refresh-token séparé.
- RLS PostgreSQL activée sur toutes les tables multi-tenant.
- Aucun secret en clair (scan gitleaks en CI).
- Revue 2-of-N sur toute PR touchant les crates efe-core ou efe-ffi.

20.4 Critères de documentation

- Chaque paquet Python a un __init__.py avec docstring de module et __all__ explicite.
- Chaque service a un runbook.md avec 10 incidents probables + procédures.
- Les ADR (Architecture Decision Records) couvrent les 12 choix majeurs (Rust core, gRPC, pure-Python stats, signing keyless, cônes, etc.).

- Les SLO sont déclarés dans docs/slo.md.
- La documentation publique (mkdocs) est publiée sur docs.efe.ai à chaque release.

Fin du document. *Ce PDF constitue la référence technique officielle du projet EFE pour la phase MVP. Il doit être relu à chaque rupture d'invariant, ajout de template d'agent, ou changement majeur de contrat gRPC. Toute question technique doit être remontée au mainteneur désigné du service concerné.*